

Testing Delivery-Conformance Checks in MP-SPDZ: A Pre-Registered Mutation Case Study

Anonymous Author(s)

Paper submitted for double-blind review

Anonymous supplementary artifact accompanies this submission

Abstract—Continuous integration turns a security property into a green check, and a green check is easily mistaken for a guarantee. We report a reproducible *testing* case study of that green check for one secure multi-party computation (MPC) application: the deterministic-query fragment of the knowledge-threshold belief computation of Mardziel, Hicks, Katz and Srivatsa (PLAS 2012), implemented on MP-SPDZ with three-party replicated secret sharing. The subject has no ground-truth oracle, so we build a separately implemented plaintext *pseudo-oracle*, an *anti-echo* interface that keeps expected outputs out of the system under test, a strict runtime transcript oracle, and recomputed, hash-bound test evidence; a bounded 228-case adversarial matrix and five committed executable negative controls exercise a static *delivery-conformance* checker that inspects compiled assembly. We evaluate that checker’s adequacy by *mutation analysis* whose operator space is a restricted, enumerated set of application-level transformations, and—because an adaptive mutation sequence adapts to the tool it hardens—we add a *pre-registered, temporally held-out* mutation study: 22 mutants with predictions committed *before* execution and run once against the frozen checker. Freezing the checker and pre-registering the corpus *reduces* post-hoc adaptation; the study remains white-box and same-project, not an independent red-team. The decisive result is a *surviving*, source-realizable mutant, H-R2: a multiset-preserving mis-delivery that the structural checker accepts and that a downstream transcript oracle catches, for that concrete labelled mutant. We report exact-strength results and honest mutant accounting (killed / survived / stillborn), and we report no aggregate mutation score for manifest-level probes whose source realizability is unproven. The artifact reproduces from a fresh checkout at the pinned backend, and the paper-critical backend observations replicate on one additional MP-SPDZ version. The contribution is in software testing; we introduce no new MPC protocol, privacy theorem, or static-analysis technique.

Index Terms—software testing, mutation testing, mutation adequacy, test oracles, pseudo-oracle, conformance testing, pre-registration, reproducibility, secure multi-party computation.

I. INTRODUCTION

An MPC framework such as MP-SPDZ [2] compiles a program into a protocol that several parties execute so each learns only its prescribed output. Building a correct *application* on top of such a framework is a separate task from the framework’s own security argument: an application author can, by accident or by an injected fault, write source that reveals a value the functionality was meant to protect. When such an application is wired into continuous integration (CI), a passing test suite is easy to mistake for evidence that the privacy property holds. This paper is a case study of *what that*

test suite can and cannot establish, told in the vocabulary of software testing: test oracles, conformance tests, and mutation adequacy.

The subject computes, per recipient, whether releasing a query’s output would let a protected party’s posterior belief exceed a public threshold, and—on accept—delivers the output *privately* to only the intended recipient. Two properties must be tested: functional correctness of the accept/reject/mask decision, and the *delivery* discipline (only party j learns $(\text{accept}_j, \text{payload}_j)$). There is no reference implementation to serve as an oracle, and the naive way to obtain one—feeding expected outputs into the harness—risks the classic *oracle-in-the-SUT* anti-pattern, where a test passes because the expected answer leaked into the system under test.

a) Contributions: The contribution is in software testing; we introduce no new MPC protocol, privacy theorem, or static-analysis technique. We contribute:

- 1) a **conformance-test design** for a subject with no trusted oracle (§III);
- 2) an **anti-echo interface** that keeps expected outputs and post-state out of the system under test (§III);
- 3) **layered functional, transcript, and evidence oracles**, each defeating a distinct way a conformance test can be vacuous (§III);
- 4) a **mutation analysis** of the compiled-delivery checker with honest killed / survived / stillborn accounting (§IV);
- 5) a **pre-registered held-out mutation protocol** that reduces post-hoc adaptation of a checker to the mutants used to harden it (§V); and
- 6) **reproducible adequacy-boundary evidence**: the layers that miss a fault and the ones that catch it, reproduced from a clean checkout (§VI, §VII).

b) Research questions: We organize the evaluation around three questions, each mapped to existing results:

RQ1 Which functional, delivery, transcript, and provenance fault classes are detected by the layered test oracles?

RQ2 Which pre-registered source-realizable held-out mutants survive the frozen delivery checker, and which downstream oracle detects them?

RQ3 Do the reported test outcomes reproduce from a clean checkout, and do the paper-critical backend observations

recur on the additional MP-SPDZ version?

II. SUBJECT AND FAULT MODELS

a) *The subject under test:* Following PLAS 2012 [1], N parties hold secrets in a finite domain D ; each observer j holds a belief over the others' secrets. For a query Q , the mechanism checks, per recipient, whether releasing Q 's output would let any protected party's posterior exceed a public threshold, quantifying over *every possible output* so the accept/reject decision is simulatable; on accept the recipient privately learns the output, on reject a masked constant, and no party may learn another's verdict. We test deterministic, total, public queries; two instantiate the subject: `pl_is_max` and `sum_even`.

b) *Backend:* Circuits are compiled and run with MP-SPDZ pinned at commit 9d809599, backend `replicated-ring-party.x`: three-party replicated secret sharing over $\mathbb{Z}_{2^{64}}$, semi-honest, honest majority. The compiler emits a manifest of *tapes* (a main tape plus comparison subtapes). Two backend facts recur and matter for the checker under test. First, a public reveal and a legitimate masked comparison-open are the *same* opcode: MP-SPDZ “uses a single instruction for opening” [2], and the correctness-check flag does not change this. Second, in the compiled paths exercised here we observed cross-tape transfer through memory and through register arguments passed by `call_tape` (opcode `0xec`) and received by `call_arg` (opcode `0xed`); a direct cross-tape register reference is a compiler error. We do *not* claim these are the only possible cross-tape channels.

c) *Runtime protocol model (out of scope for the tests):* A *protocol-level* privacy argument would target a runtime adversary—at most one semi-honest corrupted party among the three, honest majority, authenticated encrypted channels, process isolation (stated, *not* enforced by the single-host test harness). Our functional and conformance tests do *not* address it; that hand-off is §VII.

d) *Mutation model (the operator space of the study):* The mutation study injects faults into the application `.mpc` source. Because that source is compile-time Python, we state the operator space explicitly rather than “source only.” *The mutation model permits only the application-level computational and delivery transformations enumerated by the corpus. Arbitrary compile-time Python imports, filesystem or network side effects, modification of the compiler environment, and mutation of the oracle, checker, policy, CI workflow, validator, or compiler are excluded.* This is the *operator space* of the mutation analysis (§IV), *not* an adversarial-security boundary: it delimits which transformations the study applies, and the oracle, checker, and compiler are the trusted test infrastructure (Fig. 1). We make no claim under collusion, a malicious party, unencrypted channels, or a shared host, and no performance claim.

III. CONFORMANCE-TEST DESIGN

Each layer defeats a specific way a conformance test can be vacuous. We state what each layer *operationally* provides, not that it is “sound.”

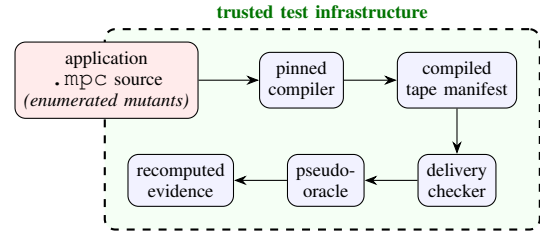


Fig. 1. Test infrastructure and the mutation operator space. The study applies the enumerated application-level transformations to the source (red); everything in the dashed box is trusted test infrastructure. The runtime protocol model (not shown) is separate and is addressed only by the verification hand-off of §VII.

a) *A separately implemented pseudo-oracle:* The test oracle is a plaintext model written from the paper, separately from the circuit and from the project’s earlier exact-rational reference model, transcribing the all-possible-outputs check directly: for each recipient it recomputes the posterior belief conditioned on *every possible* query output, accepts only if no protected party’s posterior exceeds the public threshold under any output, and delivers the actual output on accept and a masked constant on reject—the decision the circuit must match. It is a *pseudo-oracle* [3] (a separate implementation of the same specification), *not* an independently validated ground truth: it shares the specification with the circuit, so a *correlated* misreading of PLAS could make both agree on the wrong behaviour. It did carry one fatal transcription error (conditioning on the actual output only) until testing caught it. We state this shared-specification / correlated-error risk wherever we rely on oracle agreement (§VIII).

b) *An anti-echo interface:* Before any circuit was written, the interface was fixed so that expected outputs and expected post-state *never enter the system under test*—neither compiled in nor supplied as runtime inputs; they live only in the external harness, obtained from the pseudo-oracle. Circuit inputs are exactly the public parameters and the secret-shared secrets and beliefs, and the harness runs at least one state beyond the fixture, so a constant-output circuit cannot pass by accident. Beliefs travel as dense unnormalized non-negative integer weights; the threshold check avoids division ($bM \leq aZ$). Because MP-SPDZ comparison is signed, the no-wraparound requirement is $B < 2^{k-1}$ (not the naive $2^k > B$), which we pin with a counterexample test.

c) *A strict transcript oracle:* A correct party’s captured stdout must be exactly its two own records; any foreign, duplicate, unknown, missing, or extra line makes the test *fail closed*. This is an operational output-conformance oracle over the enumerated transcript cases on the observed stdout channel; it observes selected output channels and does not establish absence of every possible leak. It becomes the second line of defence in §V.

d) *Recomputed, hash-bound test evidence:* Each run retains a typed record; in CI the validator requires each record’s source hash and delivery signature to equal values recomputed from the checked-out source and a fresh compile,

and binds each record to a canonical case via a recomputed input hash and pseudo-oracle verdict. This establishes *self-consistency and provenance* of the test evidence *under the trusted CI workflow*; it is not integrity against an actor who can alter source, checker, workflow, and evidence together. It is the evidence-discipline neighbour of reproducible builds [15], recomputed per test case rather than attesting a whole artifact.

A. Test artifact and results

On the fixture (two invocations: one all-accept, one with a per-recipient reject and belief preservation) plus two further valid states, the circuit reproduces the pseudo-oracle’s visible outputs, payloads, and reconstructed weights. A *bounded, constructed* adversarial matrix of 228 cases (216 single-invocation + 12 carried; all 27 secret tuples $\times$ two queries $\times$ uniform / non-uniform / per-party-scaled integer weights near the bit bound) all match the pseudo-oracle. This is a constructed test matrix, *not* general coverage. The conformance package holds 92 collected tests—46 functional and 46 delivery-checker tests—and 9 reference test functions (25 collected items), all green; *test counts are not themselves a result*. Five committed executable negative controls are rejected by the checker (§IV). No timing is reported.

IV. MUTATION ANALYSIS OF THE DELIVERY CHECKER

Functional conformance says nothing about *how* a value is delivered: a circuit can compute correct verdicts and broadcast them. We therefore built a static delivery-conformance checker that inspects the compiled tape manifest, and evaluated its *adequacy* by mutation analysis [5], [6] over the operator space of §II. The first sequence is *retrospective and adaptively discovered*, not an independently designed operator set: the checker was repeatedly hardened against the same mutants later used as evidence, so this sequence yields no held-out adequacy estimate. §V addresses that.

a) *Mutant accounting*: One motivating transcript failure (B0); six compiled-delivery mutants (B1–B6); one semantic mutant (S1). The six are not homogeneous: B1, B2, B3 and B5 are committed executable *source* mutants; **B4 is a synthetic compiled-manifest counterexample, not a source .mpc**; and B6 is a reconstructed source mutant whose naive realization is itself *killed* (§IV-A). B0: changing `reveal_to(j)` to a public `reveal()` while keeping the per-party print compiles to a public open of every verdict, yet a stdout-only test reported success—which motivated inspecting the *compiled* delivery. B1–B3 and B5 each *survived* the then-current checker *as committed source mutants*, and each was killed by adding one detection rule, so the final rule set is the record of this progression: B1 (a separate `@function_tape` that reveals and prints) forced whole-manifest inspection rather than main-tape-only; B2 (a name-spoofed subtape doing a wrong-player delivery and a file sink) forced content rules—delivery players bound across *all* tapes, no cleartext sinks, and a strict masked-subtape name pattern; B3 (a `reveal(False)` of a verdict in a duplicate `EQZ(3)_63` via scalar memory) forced treating `False` opens as public and pinning the exact

comparison-subtape multiset; and B5 (the same via vectorized `vstms/vldms`) forced broadening the memory rule to vectorized forms. **B4** is the observation that the multiset rule is blind to tape *bodies*—a `reveal(False)` of the verdict placed in an *existing* `EQZ(3)_63` body with the three pinned names intact—demonstrated with a *synthetic manifest*, not a compiled source mutant; against the final checker it is killed by the subtape-memory rule. The clean build stores nothing in its main tape and touches no memory in its subtapes, so forbidding memory is a restriction with no false positives on the honest build.

A. B6, at its exact strength

We scope B6 precisely. We reconstructed a mutant that routes party 1’s verdict through the `call_tape/call_arg` register channel. Compiled against the pinned backend and checked by the final checker, this mutant is **KILLED**: the added tape breaks the pinned comparison-subtape multiset, and MP-SPDZ’s call frame also spills through memory, tripping the memory rules. So **B6 is not a demonstrated bypass of the final checker**; because a direct cross-tape register reference is a compile error, the B3/B5 source mutants therefore used an attacker-introduced memory hop, which the memory rules catch.

Its content is narrower. The *honest* build passes secret comparison operands across the tape boundary via `call_tape` (in main) received by `call_arg` in each of the three comparison subtapes—no memory. Therefore a detection rule that forbade cross-tape register-argument transfer of secrets, the way the memory rules close the memory channel, would **reject the honest build**: the `call_arg` channel *cannot be blocklisted by opcode/channel identity* without a false positive on the honest build. Combined with the opcode-identity fact, the testing lesson is precise: **this opcode-identity / channel-blocklist checker cannot be promoted to a general non-leakage checker**. Distinguishing legitimate from leaking flows requires analysis stronger than opcode/channel identity—operand/provenance-sensitive dataflow, typing, a verified IR, or other semantic machinery—which we did not evaluate (§IX).

B. S1: a semantic mutant the checker cannot see

A mutant that drops the reject-mask (delivers the true output even to rejected recipients) **survives** every delivery-checker rule—the delivery structure is intact—yet is **killed by the pseudo-oracle**: on secrets = (0, 0, 0), `p1_is_max`, the true output is 1 and all parties reject (oracle payload 0), but S1 delivers 1 to all three. We ran this **end-to-end at the pinned backend**: all three parties complete (return code 0) and each delivers `ACCEPT 0 / PAYLOAD 1`, an executed oracle mismatch at parties [0, 1, 2]. Delivery-conformance checks *how* values are delivered, not *which*; only the functional pseudo-oracle, bounded by test coverage, constrains that.

V. A PRE-REGISTERED, HELD-OUT MUTATION STUDY

An adaptive sequence adapts to the tool it hardens. Freezing the checker and pre-registering a temporally held-out corpus

TABLE I

MUTATION MATRIX AGAINST THE FINAL DELIVERY CHECKER (MACHINE-READABLE FORM IN THE SUPPLEMENTARY ARTIFACT; OUTCOMES RECOMPUTED AGAINST THE FROZEN CHECKER AT A SINGLE PINNED MP-SPDZ COMMIT). “KILLED” = CHECKER REJECTS; THE FIVE COMMITTED CONTROLS MAP TO B0–B3 AND B5. B4 IS A *synthetic* COMPILED-MANIFEST COUNTEREXAMPLE, NOT A SOURCE .MPC; B6 AND S1 ARE RECONSTRUCTED SOURCE MUTANTS. NOT COMMITTED TO THE FROZEN ARTIFACT: B4, B6, S1.

ID	Committed?	Mutation / channel	Outcome	Notes
B0	leaky	public reveal() in main	KILLED	motivating stdout failure
B1	subleak	separate @function_tape reveal+print	KILLED	killed by whole-manifest inspection
B2	namespoof	name-spoofed subtape: wrong-player delivery + file sink	KILLED	killed by content rules
B3	openfalse	reveal(False) in spoofed EQZ(3)_63, scalar mem	KILLED	False open is still public
B4	<i>synthetic</i>	reveal(False) in an <i>existing</i> EQZ(3)_63 body, multiset intact	KILLED (rule h)	synthetic manifest; not a source .mpc
B5	openfalse_vec	same via vectorized vstms/vldms	KILLED	killed by broadened regex
B6	<i>reconstructed</i>	verdict via call_tape/call_arg register args	KILLED (naive)	see §IV-A; not a checker bypass
S1	<i>reconstructed</i>	correct delivery, reject-mask dropped	SURVIVES checker / KILLED by oracle	semantic mutant; checker is semantics-blind

reduces post-hoc adaptation, although the study remains white-box and same-project rather than an independent red-team evaluation. We committed a corpus of 22 mutants with locked predictions *before* running them, then ran once against the frozen checker; we report per-category outcomes and *no* aggregate mutation score.

A. Mutation operators and the pre-registration protocol

The 22 mutants instantiate seven operator categories, drawn from three sources so the corpus is not a renaming of B0–B6: security mutation-testing practice (systematic operator families, and separating *stillborn* and *equivalent* mutants from killed ones [6], [7]); the information-flow observation that leakage is a dataflow rather than opcode-identity property, which *predicts* surviving value-routed and permutation mutants (§IX); and MP-SPDZ backend mechanisms (the open family, the `call_tape/call_arg` register channel, memory forms, sink opcodes, and the cross-tape compile error). The categories are: (1) recipient/destination; (2) public/private open; (3) subtape/topology; (4) secret-transfer/channel; (5) output/value-semantic; (6) transcript/output-channel; and (7) evidence/provenance (which targets the *separate* evidence validator, not the delivery checker). Concretely, the corpus exercises forms the adaptive sequence never used: indirect (`ldmsi`) and `gf2n` memory loads inside a subtape, network (`writesocket`) and file (`writesharestofile`) sinks, and numeric (`print_int`) prints—all killed by the checker’s rules—alongside a register-argument open of a passed value in an existing masked subtape (the H-O3 probe), which no rule fires on and which therefore survives at the manifest level. Table II lists the operator categories, the transformation each applies, the source/synthetic split, and the headline outcome per category.

The protocol is two-phase and its integrity is visible in the version history. First, the corpus, the mutant sources or syn-

thetic transforms, and a *locked prediction* for every mutant—predicted compile outcome, predicted checker verdict, source realizability, and denominator membership—are committed *before* the checker is run on them. Second, the study is executed *once* and the machine-readable results are committed separately. We then classify each mutant honestly: *killed* (checker rejects), *survived* (checker accepts), *stillborn* (does not compile; never counted as a kill), and *benign/equivalent* (correctly not flagged). We record per mutant whether it is *source-realizable* (a compiled .mpc) or a *synthetic manifest-level probe* whose source realizability is unproven, and we report **no aggregate mutation score over the synthetic probes**: a surviving probe shows only that no rule fires on that assembly pattern, not that a source program can produce it (B6 shows the naive realization is killed). All 22 outcomes matched their pre-registered predictions—evidence about the checker’s *rules*, not about an independent adversary’s ingenuity.

B. The decisive surviving mutant (H-R2)

Among three pre-registered source-realizable recipient-structure mutants, the checker killed H-R1 and H-R3 and *survived* the multiset-preserving H-R2 permutation. H-R2 rotates the delivery target so party j ’s verdict is delivered to player $r = (j+1) \bmod N$ while the per-party record label is unchanged:

```
for j in range(N):
    r = (j + 1) % N
    print_ln_to(r, "PRIV %s ACCEPT %s", j,
                accept[j].reveal_to(r))
    print_ln_to(r, "PRIV %s PAYLOAD %s", j,
                payload[j].reveal_to(r))
```

The same permutation applies to both the ACCEPT and the PAYLOAD record. Table III shows why it survives the structural checker: the destination-player multiset $[0, 0, 1, 1, 2, 2]$ —all the main-tape delivery rule inspects—is preserved, so the checker cannot see that the value→recipient *pairing* is

TABLE II

MUTATION-OPERATOR CATEGORIES IN THE HELD-OUT CORPUS (22 MUTANTS). “SRC” = SOURCE-REALIZABLE (COMPILED .MPC); “SYN” = SYNTHETIC MANIFEST-LEVEL PROBE (SOURCE REALIZABILITY UNPROVEN); “PROV” = EVIDENCE TRANSFORM SCORED AGAINST THE SEPARATE VALIDATOR.

Category	Transformation applied	src/syn	Headline outcome
recipient/destination	who receives each verdict (main-tape delivery)	3/0	H-R2 <i>survives</i> (multiset preserved); H-R1,H-R3 killed
public/private open	raw / <code>False</code> / vectorized opens via the register channel in a subtape	0/3	H-O3 <i>survives</i> (no rule); H-O1,H-O2 fire
subtape/topology	reorder, add, or rename comparison subtapes	0/3	benign reorder not flagged; off-pin / extra tape fires
secret-transfer/channel	memory forms (indirect <code>ldmsi</code> , <code>gf2n</code>) and legitimate <code>call_arg</code>	0/3	new memory forms fire; legitimate <code>call_arg</code> not flagged
output/value-semantic	correct delivery structure, wrong delivered value	3/0	all <i>survive</i> checker; killed by the pseudo-oracle
transcript/output-channel	file / socket / numeric-print sinks in a tape	0/3	all fire (killed)
evidence/provenance	forged / replayed test evidence	0/2 (prov)	rejected by the separate evidence validator
(<i>compile-invalid</i>)	<i>direct cross-tape reference; undefined symbol</i>	2 <i>src</i>	<i>stillborn; never counted as a kill</i>

TABLE III

H-R2 ACROSS THE LAYERED ORACLES. THE STRUCTURAL CHECKER IS BLIND TO A MULTISSET-PRESERVING MIS-DELIVERY; A DOWNSTREAM TRANSCRIPT ORACLE KILLS THIS CONCRETE MUTANT.

Layer	H-R2 outcome
Destination multiset	preserved
Delivery checker	survives
Transcript oracle	killed
Reason	foreign <code>PRIVj</code> record on the receiving stream
Scope	this concrete labelled permutation only

wrong. It is killed only downstream, by the strict transcript oracle, because on player *r*’s stream a foreign `PRIVj` record appears. We do *not* claim the transcript oracle catches recipient permutations in general; it kills *this concrete labelled mutant*.

Three source-realizable *semantic* mutants (negated accept, swapped accept/payload, a neighbour’s payload) likewise survive the checker and are killed by the pseudo-oracle, echoing S1. The remaining *synthetic* mutants are *manifest-level rule-coverage probes*—hand-built assembly patterns testing whether a rule fires, with *unproven* source realizability, *not* executable mutants, and excluded from any mutation-score denominator. The informative probe is H-O3: a `call_arg` open of a register-passed value in an existing masked subtape that no current rule rejects—the same gap B6 names (§IV-A), now exhibited at the manifest level. Table IV summarizes the outcome classes.

VI. REPRODUCTION AND CROSS-VERSION REPLICATION

The frozen artifact reproduces from a fresh checkout with a fresh build of the pinned MP-SPDZ backend: 117 collected `pytest` items (the 92 conformance items plus the 9 reference test functions, which expand to 25), the named-case harness 4/4, the 228-case matrix 228/228, the recomputed evidence, and the B4/B6/S1 reproductions all reproduce, and the normalized per-tape `manifest_signature` values recompute to the archived values *at the pin*. We reproduce those *normalized-assembly* signatures, *not* raw binaries or assembly byte-for-byte. Separately, the paper-critical backend

observations replicate on one further MP-SPDZ version, v0.4.3 (26a60536): the shared open-instruction family, the honest `call_tape/call_arg` path, the B6 behaviour, and S1’s structural survival with an executed oracle mismatch all recur, with no compile/API or semantic change in what we exercised. This is two versions, not a cross-version compatibility or version-independence claim, and we do *not* claim the compiled-assembly signatures reproduce across versions: only one additional version, on one host, for the probed observations, was tested. A one-command reproduction wrapper and machine-readable results are in the supplementary artifact.

A. Results summary by research question

RQ1. The layered oracles detect the gross delivery-fault classes the checker has rules for—public opens, illicit sinks, disallowed memory, off-pin tapes (Table I, and the “rule fires” rows of Table IV)—and the pseudo-oracle detects value/semantic faults the delivery checker cannot see (S1; H-V1–H-V3); the provenance mutants (H-E1, H-E2) are rejected by the separate evidence validator. **RQ2.** Of three pre-registered source-realizable recipient-structure mutants the checker kills H-R1 and H-R3 and *survives* H-R2; H-R2 is detected downstream by the *transcript oracle* for that concrete labelled mutant (§V-B); the semantic survivors H-V1–H-V3 are detected by the *pseudo-oracle*. **RQ3.** All reported outcomes reproduce from a clean checkout at the pin, and the four paper-critical backend observations recur on the additional MP-SPDZ version; we make no stronger cross-version claim.

VII. WHAT THE TEST SUITE ESTABLISHES

The assurance is a chain of partial checks, no one of them a privacy proof. The pinned *compiler* rejects one naive cross-tape transfer outright (a direct cross-tape register reference is a compile error; the B3/B5 source mutants therefore used an attacker-introduced memory hop). The *delivery checker* kills gross structural and channel mutant classes, including forms outside its design set (exercised as manifest-level probes, not executable source mutants), yet H-R2 shows it survives a multiset-preserving wrong-recipient *structure*. The *transcript oracle* killed that one concrete mutant; the *pseudo-oracle* kills

TABLE IV

HELD-OUT MUTANT OUTCOME CLASSES (22 MUTANTS, SINGLE RUN AGAINST THE FROZEN CHECKER; PREDICTIONS PRE-REGISTERED BEFORE EXECUTION). “SURVIVES” = CHECKER PASSES. NO AGGREGATE MUTATION SCORE IS REPORTED FOR THE MANIFEST-LEVEL PROBES, WHOSE SOURCE REALIZABILITY IS UNPROVEN.

Class	Mutants	Outcome
source-realizable, recipient	H-R1, H-R3	killed by checker
source-realizable, recipient	H-R2	survives checker; killed by transcript oracle (this mutant)
source-realizable, semantic	H-V1–H-V3	survive checker; killed by pseudo-oracle
synthetic manifest probe	H-O1,H-O2,H-T2,H-T3,H-C1,H-C2,H-P1–H-P3	rule fires (probe “killed”)
synthetic manifest probe	H-O3	no rule fires (<i>survives</i>); realizability unproven
synthetic (benign)	H-T1, H-C3	correctly not flagged
stillborn (compile-invalid)	H-X1, H-X2	do not compile; not a kill
provenance (other checker)	H-E1, H-E2	rejected by the evidence validator

semantic mutants delivery-checking survives; and *evidence recomputation* protects the consistency and provenance of these records under the trusted CI. In the tested examples, later layers catch some faults earlier checks leave open; the layers are complementary, not complete, and none, alone or together, is a protocol-level privacy proof.

a) *What would make it a guarantee*: Two verification obligations, neither supplied by our tests: (i) *privacy of the executed circuit*—a simulation argument or a formally-verified, dataflow-checked comparison primitive; and (ii) *semantic source-to-spec binding*—that the circuit computes the intended function, not merely one that agrees with the pseudo-oracle on tested cases (S1 shows why delivery privacy is insufficient without it). Our tests give coverage-bounded evidence toward (ii) and nothing toward (i).

VIII. THREATS TO VALIDITY

- **Construct validity (mutation operator space)**. The mutation study permits only the enumerated application-level computational and delivery transformations; arbitrary compile-time Python imports, filesystem or network side effects, compiler-environment modification, and mutation of the oracle, checker, policy, CI, validator, or compiler are excluded. Results are stated over this operator space, not an unrestricted adversary.
- **Correlated authorship and AI assistance**. The implementation, tests, review synthesis, and manuscript were produced within one project with substantial AI assistance; model agreement is not independent validation.
- **Adaptive overfitting, partially mitigated**. The checker was hardened against the same mutants later used as the first evidence. Freezing the checker and pre-registering a temporally held-out corpus (§V) *reduces* post-hoc adaptation, but the study is white-box and same-project, not an independent red-team; we report per-category outcomes rather than a mutation score.
- **Pseudo-oracle pseudo-independence**. The oracle shares PLAS with the circuit; a correlated spec misreading is possible. A cross-check against the project’s exact reference model is limited because that model implements a

different functionality; we state the risk rather than claim it eliminated.

- **External validity**. “Conformance” means agreement with a pseudo-oracle on tested inputs. Results are for one deterministic-query functionality, $N=3$, $D=\{0, 1, 2\}$, one backend, and a single-host harness that is not an adversarially isolated deployment; the backend-level observations were additionally checked on one further MP-SPDZ version, which is two versions, not a cross-version compatibility claim.
- **No performance or simulation-security claim**; the boundary result is a case-study observation, not a theorem.

IX. RELATED WORK

Novelty statements below rest on a bounded, documented search; they are “to the best of a bounded search,” not absolute.

a) *Test oracles and pseudo-oracles*: The oracle problem—deciding the expected result of a test—is surveyed by Barr et al. [3]. When no ground truth exists, a pseudo-oracle (a second implementation of the same specification) and metamorphic testing [4] are two standard responses; both address oracle absence, but this study uses a separately implemented pseudo-oracle rather than metamorphic relations across executions. Our anti-echo interface is a discipline for avoiding the oracle-in-the-SUT trap; Wycheproof applies reference-oracle conformance to crypto *primitives* by feeding expected outputs, the opposite of our anti-echo design.

b) *Mutation testing and negative controls*: Mutation testing dates to DeMillo et al. [5] and is surveyed by Jia and Harman [6]; whether mutants substitute for real faults is studied by Just et al. [7]. We use mutants as an *adequacy probe for a checker* and are careful about mutant classes—killed, survived, stillborn (compile-invalid), and benign/equivalent—never counting a stillborn mutant as a kill. Closest in spirit, MASC [8] evaluates crypto-misuse detectors with planted leaky mutants, but as an offline third-party-detector benchmark rather than committed CI controls; and, unlike a standard mutation-score study, we *pre-register* a held-out corpus to reduce the adaptive coupling between a checker and the mutants used to harden it.

c) *Leakage is dataflow, not opcode identity*: `ct-verif` reduces constant-time to a 2-safety property on LLVM IR [9]; `Binsec/Rel` decides constant-time *relationally at the binary level* on operand provenance [10]; `Securify` decides EVM-bytecode security on inferred *dataflow* facts rather than syntactic patterns [11]. Our §IV-A instantiates this principle in the MPC-application testing setting; we claim the instantiation and the pinned-backend mechanism, not the principle. The positive dual is language-based security for MPC: Kerschbaum’s information-flow type system [12], and, closest to our finding, Skalka and Near’s *sound* static security-type system for a purpose-built low-level MPC IR [13], [14], which is *sound because* its IR makes declassification syntactically distinct—the property MP-SPDZ bytecode lacks. Reproducible Builds [15] is the evidence-discipline neighbour.

X. CONCLUSION

On one MP-SPDZ application, a separately implemented pseudo-oracle, an anti-echo interface, a strict transcript oracle, and recomputed evidence give real, coverage-bounded, operationally-stated test assurance and kill a family of gross delivery-fault classes—and they have a sharp, nameable adequacy boundary. A static opcode-identity / channel-blocklist checker cannot be promoted to a general non-leakage checker: a masked open and a raw reveal are the same instruction, and the register channel a leak would use is the one the honest code uses. A pre-registered, held-out mutation study makes the boundary concrete with a *surviving* source-realizable mutant (H-R2, a multiset-preserving mis-delivery) caught only by a downstream oracle. Distinguishing the flows needs analysis stronger than opcode/channel identity—which we did not evaluate—and a privacy guarantee needs a protocol-level proof. Naming that boundary—and printing it next to the green check—is the case study’s point.

ARTIFACT AVAILABILITY

The system under test, the pseudo-oracle, the frozen delivery checker, the five committed negative controls, the mutation corpus and machine-readable mutation matrix, the pre-registered held-out corpus with its locked predictions and single-run results, and one-command reproduction wrappers are provided as *anonymized supplementary material*. The pre-registration and the results are separate, time-ordered commits; the conformance artifact is frozen at a single commit. All quantitative claims reproduce from a fresh checkout with a fresh build of the pinned MP-SPDZ backend; reproduction uses a git clone because the wrappers check out the frozen commit.

REFERENCES

- [1] P. Mardziel, M. Hicks, J. Katz, and M. Srivatsa, “Knowledge-Oriented Secure Multiparty Computation,” in *Proc. ACM SIGPLAN Workshop on Programming Languages and Analysis for Security (PLAS)*, 2012.
- [2] M. Keller, “MP-SPDZ: A Versatile Framework for Multi-Party Computation,” in *Proc. ACM CCS*, 2020, pp. 1575–1590.
- [3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The Oracle Problem in Software Testing: A Survey,” *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, 2015.
- [4] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, “Metamorphic Testing: A Review of Challenges and Opportunities,” *ACM Comput. Surv.*, vol. 51, no. 1, art. 4, pp. 4:1–4:27, 2018.
- [5] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Hints on Test Data Selection: Help for the Practicing Programmer,” *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [6] Y. Jia and M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, 2011.
- [7] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are Mutants a Valid Substitute for Real Faults in Software Testing?” in *Proc. ACM SIGSOFT FSE*, 2014, pp. 654–665.
- [8] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyanyk, and A. Nadkarni, “Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques,” in *Proc. IEEE S&P*, 2022.
- [9] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, “Verifying Constant-Time Implementations,” in *Proc. USENIX Security*, 2016.
- [10] L.-A. Daniel, S. Bardin, and T. Rezk, “Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary Level,” in *Proc. IEEE S&P*, 2020.
- [11] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, “Securify: Practical Security Analysis of Smart Contracts,” in *Proc. ACM CCS*, 2018.
- [12] F. Kerschbaum, “An Information-Flow Type-System for Mixed Protocol Secure Computation,” in *Proc. ACM ASIACCS*, 2013.
- [13] C. Skalka and J. P. Near, “Language-Based Security for Low-Level MPC,” in *Proc. PPDP*, 2024.
- [14] C. Skalka and J. P. Near, “SMT-Boosted Security Types for Low-Level MPC,” in *Proc. ESOP, LNCS 15695*, 2025, pp. 258–285.
- [15] C. Lamb and S. Zacchiroli, “Reproducible Builds: Increasing the Integrity of Software Supply Chains,” *IEEE Software*, 2022.